Soccer Stars ASIC

Elec 422 Final Project Report

May 5, 2026

Team Members

Arda Inegol

Ioan-Alexandru Mirica

Abstract

This ASIC implements a player-versus-computer penalty shootout game inspired by the mechanics of games like FIFA. The player latches power, spin, and horizontal/vertical angle values generated by ring oscillators by pressing the corresponding control inputs. The final ball position is computed using a shift-add multiplier, while the goalkeeper position is generated via an 8-bit LFSR pseudo-random number generator. The ball and keeper coordinates are compared to determine a save or a goal, producing a win or loss indication. The chip is intended for use with an LED matrix, where separate colors indicate the location of the ball and goalkeeper.

Contents

Introduction and Motivation 2

Design Partitioning 2

FSM Controller 3

Datapath 5

System Integration 6

Two-Phase Clocking 7

Padframe Integration 8

Verification and Testing 10

Results and Lessons Learned 12

Conclusion and Future Testing 13

Appendix 14

Introduction and Motivation

As designers of the Soccer Stars ASIC, we drew inspiration from the FIFA game series and aimed to capture a similar feel within a realistic design scope. Rather than attempting a full soccer simulation, which would be far more complex, we focused on a penalty shootout scenario that still delivers engaging gameplay while remaining feasible to implement. The game is intended for a broad audience, appealing to both soccer fans and anyone who enjoys simple yet sufficiently interactive experiences.

Project Goals

- Create FSM controller handling state transitions with at least 10 states
- Create datapath handling main logic and two phase timing parameters
- Successfully implement pseudo-number generation with an LFSR
- Successfully implement ring oscillator functionality for input latching
- Combine synthesized core with padframe and perform testing from padframe

Software Used

- Design Compiler $\rightarrow$ Verilog module synthesis
- Magic VLSI $\rightarrow$ .ext, .mag, .sim and .cif file creation
- Cadence Innovus $\rightarrow$ .gds file creation
- Siemens QuestaSim $\rightarrow$ before and after synthesis simulation
- IRSIM $\rightarrow$ post routing simulation

Language Used

- Verilog

Design Partitioning

Following the class structure, and especially the structure emphasized in Homeworks 2 and 3, we split the design into two main synthesizable modules, such that fsm_controller handles all sequencing and state transitions, and datapath holds every register, multiplier, and arithmetic block. A third module, soccer_stars_top, instantiates both, generates the four ring-oscillator inputs, and routes the inter-module control bus. This separation is optimal as it means the FSM never directly manipulates data values and the datapath never decides when to act, only how to act. The partition also aligns naturally with the two-phase clocking scheme, since both modules share the same clka and clkb convention.

Module Hierarchy

- **soccer_stars_top:** Top-level module that instantiates fsm_controller and datapath, wires the control bus between them, and generates the four 4-bit ring-oscillator values (ring_power, ring_spin, ring_v_angle, ring_h_angle) from free-running counters with co-prime increments (1, 3, 5, 7) so the rings do not track each other. The module also exposes player inputs, result outputs, and signals which help debugging.
- **fsm_controller:** This module is essentially a 15-state Moore machine that sequences the game from input capture through trig lookup, the three multiplications, keeper sampling, collision check, and display. It drives all enable signals and the multiplication_select bus.
- **datapath:** The datapath holds the input registers, trig ROM, shared shift-add multiplier, physics result registers (v_sq, range, lateral), LFSR, keeper sampler, and collision/score logic. It also calculates the final ball and keeper coordinates plus goal_flag, which gets exposed at the top level as win_loss, and valid.

Inter-Module Interfaces

We ensured that the FSM and datapath communicate exclusively through control signals flowing from fsm_controller to datapath, meaning no data flows back. Branch conditions (strike_power_ctrl, spin_ctrl, v_angle_ctrl, h_angle_ctrl, enable, restart) are sampled by the FSM directly from top-level inputs rather than from the datapath, which keeps the interface one-way and avoids any combinational loops between the two modules. The four ring-oscillator buses generated in soccer_stars_top are passed straight into the datapath

and are only sampled when the corresponding store-enable signals is asserted. Below are all the control signals, their widths in bits and their main functionalities.

Control Signals

Signal	Width (bits)	Function
strike_store_enable	1	Latches ring_power into power_reg
spin_store_enable	1	Latches ring_spin into spin_reg
v_angle_store_enable	1	Latches ring_v_angle into v_reg
h_angle_store_enable	1	Latches ring_h_angle into h_reg
access_trig_enable	1	Registers trig ROM outputs and computes spin_adj
multiplication_select	2	Selects multiplier operation (01: power^2, 10: range, 11: lateral)
sample_keeper_enable	1	Loads keeper_x and keeper_y from LFSR and inputs
range_enable	1	Adds spin_adj into range_reg
lateral_enable	1	Marks lateral computation active
collision_check_enable	1	Triggers compare and updates goal_flag
display_enable	1	Asserts valid for display

FSM Controller

The FSM controls input loading, computation, and display. It begins in IDLE, waiting for enable. It then sequentially loads inputs through LOAD_STRIKE, LOAD_SPIN, LOAD_V_ANGLE, and LOAD_H_ANGLE, each advancing when its control signal is asserted. Next, LOOKUP_TRIG retrieves trig values. CALC_V_SQ, CALC_RANGE, and CALC_LATERAL perform iterative computations using counters. APPLY_SPIN adjusts the range, and GOALKEEPER generates a random keeper position. COLLISION checks overlap. Finally, DISPLAY_SHOT and DISPLAY_KEEPER drive the output, and OUTPUT_VALID holds the result until reset. In total, there are 15 states present. Below we are presenting a simple block diagram of the module and then a complete state transition diagram, as well as a short description of each state.

FSM Block Diagram

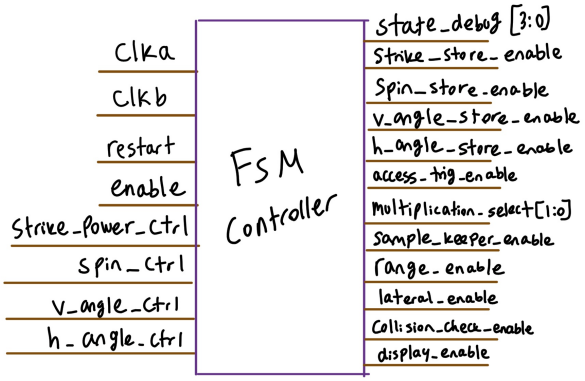

Figure 1: Block diagram of FSM controller.

State Transition Diagram

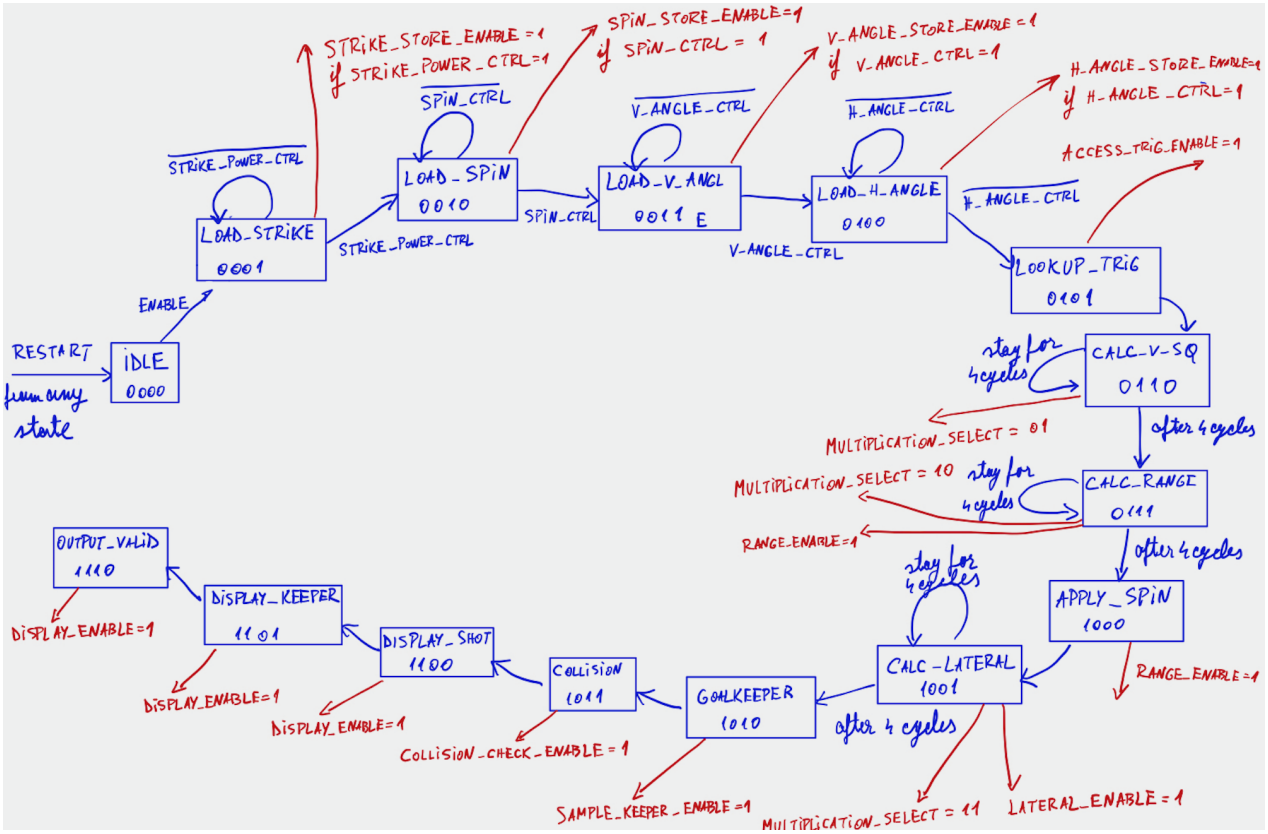

Figure 2: State transition diagram of FSM controller states.

State Descriptions

State	Description	Outputs / Actions
IDLE	Wait for enable signal to start operation	No outputs asserted
LOAD_STRIKE	Wait for strike input to be confirmed	strike_store_enable
LOAD_SPIN	Wait for spin input to be confirmed	spin_store_enable
LOAD_V_ANGLE	Wait for vertical angle input	v_angle_store_enable
LOAD_H_ANGLE	Wait for horizontal angle input	h_angle_store_enable
LOOKUP_TRIG	Perform trig lookup for angles	access_trig_enable
CALC_V_SQ	Iteratively compute velocity squared	multiplication_select active
CALC_RANGE	Iteratively compute shot range	range_enable
APPLY_SPIN	Apply spin correction to range	spin_store_enable
CALC_LATERAL	Iteratively compute lateral position	lateral_enable
GOALKEEPER	Generate goalkeeper position (LFSR)	sample_keeper_enable
COLLISION	Check collision between ball and keeper	collision_check_enable
DISPLAY_SHOT	Display ball position	display_enable
DISPLAY_KEEPER	Display keeper and overlap result	display_enable
OUTPUT_VALID	Hold final result indefinitely	display_enable

Transition Logic

State transitions are primarily driven by input control signals, internal counters, and fixed sequencing. Loading states advance only when their corresponding control signal is asserted; otherwise, they stall. Computation states (e.g. CALC_RANGE) loop until their counters reach a set value. Remaining states transition unconditionally in sequence.

Datapath

The datapath takes four 4-bit user inputs (power, spin, vertical angle, horizontal angle) and computes a ball trajectory, compares it against a pseudo-randomly placed goalkeeper, and produces a goal/no-goal result. It is split across two clock domains: clka drives input capture, the trig lookup, and the multiplier shift registers, while clkb drives the accumulator, the physics result registers, the keeper sampler, and the collision logic.

Inputs and trig outputs are 4 bits, giving a 16×16 playfield. Products of two 4-bit values can reach 8 bits, so mult_accum, v_sq_reg, range_reg, and lateral_reg are 8 bits. Final ball and keeper coordinates are truncated to 4 bits for the collision check. The LFSR is 8 bits and the score counter is 3 bits. Below we explain the more specialized we had to create to handle input latching and pseudo-random number generation.

Special Circuit Blocks

LFSR pseudo-random number generator: An 8-bit Fibonacci LFSR provides the randomness for the goalkeeper position. Each cycle it shifts right by one bit and XORs selected tap bits back into the input, producing a long pseudo-random sequence from very little hardware. We use a tap mask of 0x70, XORing bits 4, 5, and 6 into the new bit. On restart it is seeded to 0xAA to avoid the all-zero lock-up state, where the LFSR would get stuck producing zeros forever. It free-runs on ϕ_1, so the value sampled into keeper_x and keeper_y depends on how long the user takes to press the inputs, creating variation in the gameplay.

Ring oscillator-like counters: Rather than full inverter-chain ring oscillators, the four ring inputs are modeled as free-running 4-bit counters with co-prime increments (1, 3, 5, 7) so each sequence advances at a different rate. In a physical implementation these would be replaced by actual ring oscillators, but in our flow Design Compiler would synthesize a true inverter ring away, so counters were necessary. The behavior is preserved regardless as each ring still produces a fast-changing value that the player latches, and the counters are deterministic for simulation.

Datapath Block Diagram

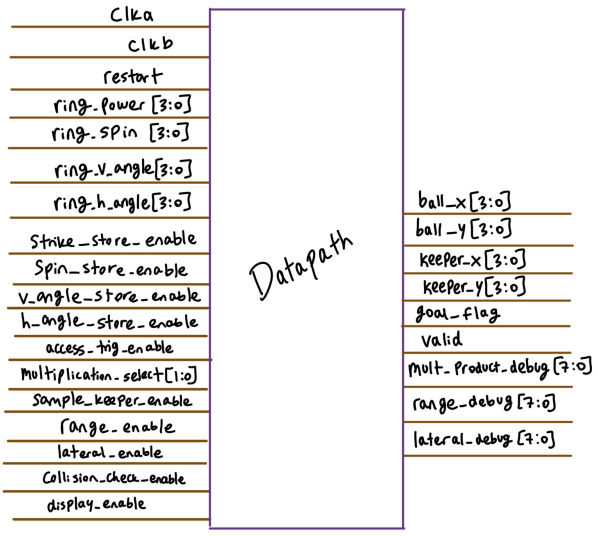

Figure 3: Block diagram of datapath module.

Datapath Components

Component	Width (bits)	Function
power_reg, spin_reg, v_reg, h_reg	4	Latch ring inputs on store-enable
Trig ROM, trig_v, trig_h	4	16-entry lookup, registered on access_trig_enable
spin_adj	4	spin_reg shifted right by 1; feeds range and lateral
Shift-add multiplier	8	Shared 5-cycle multiplier selected by multiplication_select
v_sq_reg, range_reg, lateral_reg	8	Capture multiplier result for each operation
LFSR	8	Free-running pseudo-random source for keeper
keeper_x, keeper_y	4	XOR of LFSR halves with user inputs
Collision logic	—	Compares ball and keeper with tolerance 2
score	3	Increments on each goal
valid	1	Asserted by display_enable when outputs stable

System Integration

We decided to wire the FSM and datapath together inside soccer_stars_top, which acts as the chip's top-level shell. The top module instantiates one fsm_controller (u_fsm) and one datapath (u_datapath), declares an internal control bus of eleven enable wires, and routes them from the FSM's outputs to the datapath's matching inputs. Both modules are driven by the same clka and clkb so the two-phase scheme stays consistent across the partition, and both receive the same restart line so the chip resets as a single unit.

Beyond instantiation, the top module also generates the four ring-oscillator buses that feed the datapath's input registers. Four free-running 4-bit counters with co-prime increments (1, 3, 5, 7) advance on every negedge of clka and are exposed as ring_power, ring_spin, ring_v_angle, and ring_h_angle. The player latches each value by asserting its corresponding control signal, which the FSM uses both as a branch condition to advance to the next load state, while also activating matching store-enable for one cycle.

FSM and Datapath Integration Block Diagram

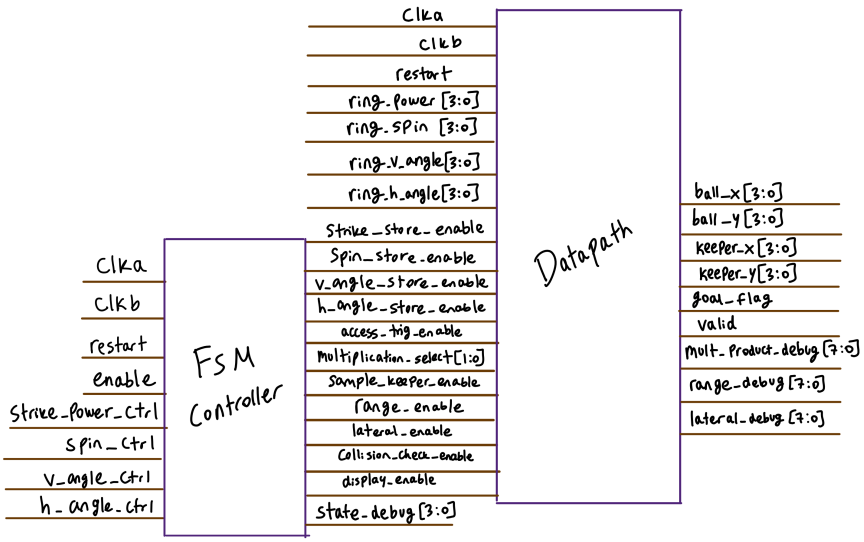

Figure 4: Block diagram of connected FSM and datapath modules.

Synthesized Core Layout

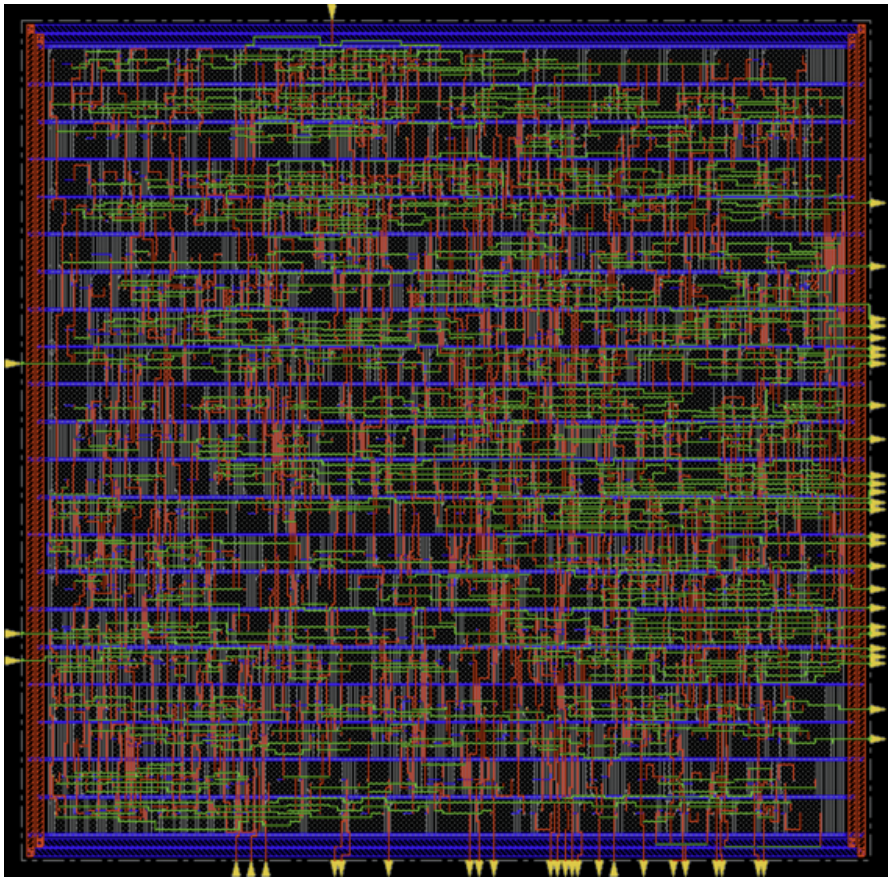

Figure 5: Innovus layout of the synthesized core using FSM, datapath and top modules.

Reset and Initialization

On power-up or whenever restart is asserted, every register in both modules is cleared synchronously on the next negedge. As expected, we ensured that the FSM enters its IDLE state with all outputs deasserted, and the datapath clears its input registers, multiplier shift registers, accumulator, physics registers, keeper, score, and goal_flag. The LFSR is the one exception since it is seeded to 0xAA on restart, so it never enters the all-zero lock-up state. The four ring-oscillator counters in the top module also reset to zero and immediately resume free-running on the next clka edge.

Once restart is released, the FSM holds in IDLE until enable is asserted, at which point it advances to LOAD_STRIKE and begins waiting for the player's first control input. Each subsequent load state waits for its matching control signal before advancing, so the player paces the input phase. After all four inputs are captured, the FSM proceeds automatically through trig lookup, the three multiplications, keeper sampling, collision check, and display without further user input. To start a new round, the player reasserts restart, which returns the entire chip to its initial state.

Two-Phase Clocking

The design uses two non-overlapping clocks, clka (ϕ_1) and clkb (ϕ_2), to separate input and multiplier setup from result accumulation and output. All registers are triggered on negedge. Because the two clocks never assert simultaneously, a value written by one phase is guaranteed to be stable before the other phase samples it, which removes any issues that might occur between the multiplier shift registers and the accumulator.

We chose two-phase clocking per the assignment instructions but also because the multiplier produces a new

partial_product every clka cycle that must be summed into mult_accum on clkb. Driving both with a single edge would require significant optimization through for instance a pipeline register, and our non-overlapping scheme avoids this need. Below we provide a diagram showing how the clocking scheme functions and then list what operation is performed at each clock phase.

Clock Phase Diagram

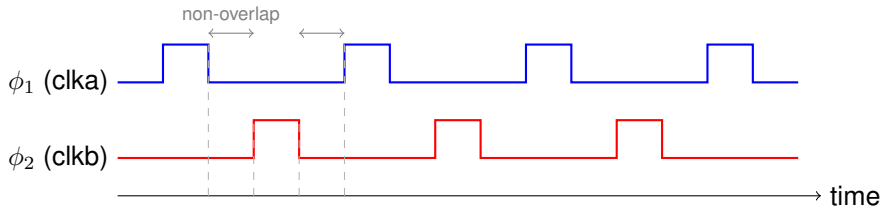

Figure 6: Two-phase non-overlapping clock waveforms such that ϕ_2 rises only after ϕ_1 has fallen, guaranteeing a dead window between phases.

Phase Assignments

Element	Clock Phase	Purpose
Input registers	ϕ_1 (clka)	Capture ring inputs before computation
Trig registers	ϕ_1 (clka)	Hold ROM lookups stable for multiplier
Multiplier shift registers	ϕ_1 (clka)	Advance one shift per cycle, produce partial product
LFSR	ϕ_1 (clka)	Free-runs to generate keeper randomness
Accumulator	ϕ_2 (clkb)	Sums partial products from ϕ_1
Physics registers	ϕ_2 (clkb)	Latch multiplier result on cycle 5
Keeper registers	ϕ_2 (clkb)	Sample LFSR after ϕ_1 settles
Output stage	ϕ_2 (clkb)	Drive collision result and outputs

Padframe Integration

With 54 pins on our core, we used the provided 64-pin padframe to host the design. We opened the .gds in Magic, routed each core pin to its corresponding pad, and relabeled the pads as p_[pin_name] for clarity.

To meet the minimum density requirements (15% poly, 30% metal1, and 30% metal2) we added fill on each of those layers inside the padframe. After flattening the layout and rechecking densities, every layer measured above its required threshold. Specifically, we had that poly = 0.152 > 0.15, metal1 = 0.301 > 0.30, and metal2 = 0.309 > 0.30.

Below is the final integrated circuit diagram with the core placed inside the padframe and extra layers inside to satisfy density requirements. Moreover, each pad is labeled with its hosted pin and marked as an input or output, as requested in the instructions. The image below the circuit confirms that all layer densities exceed their thresholds.

Final Integrated Circuit Layout and Pinout

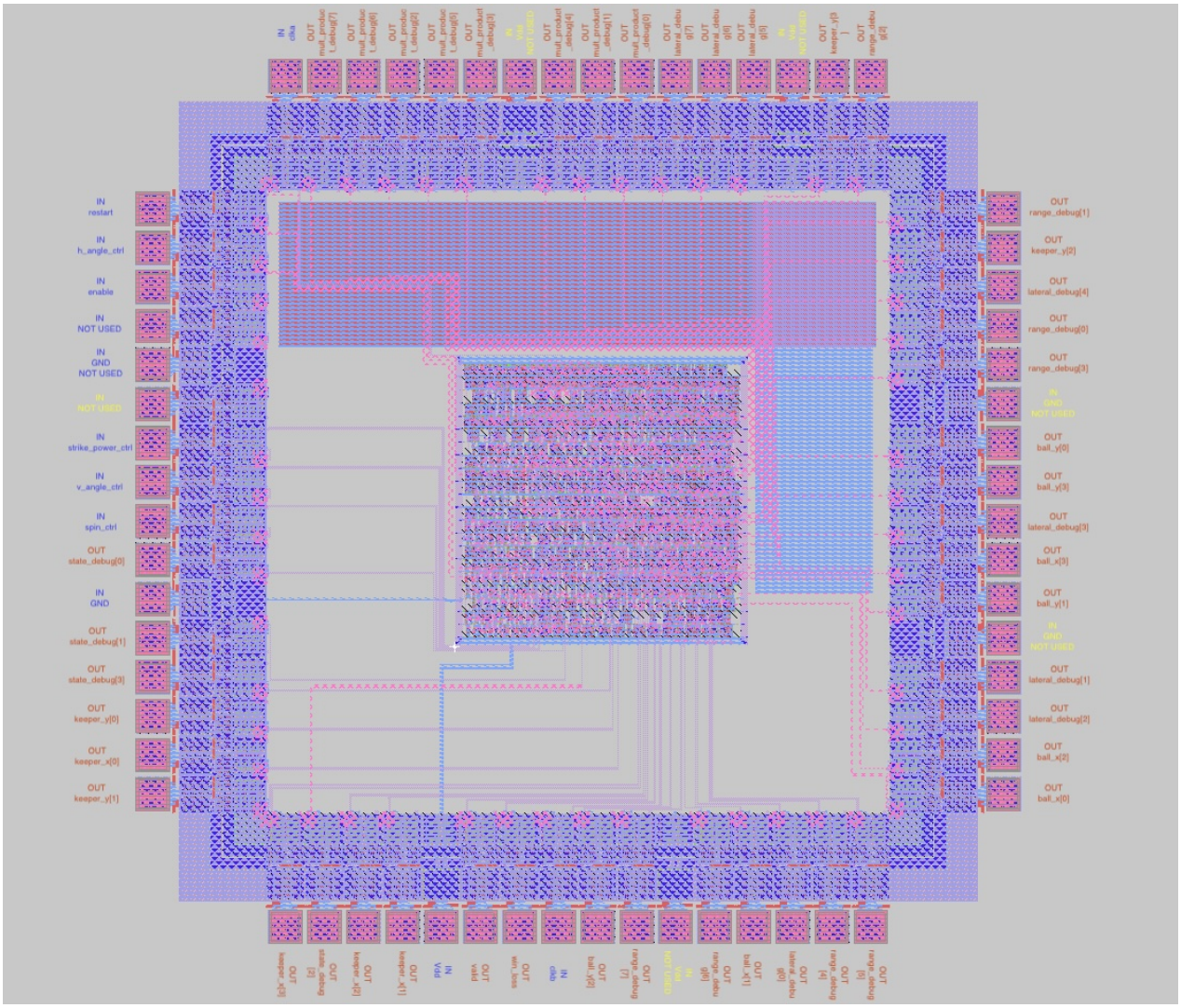

Figure 7: Integrated core and padframe.

Final Layer Densities

```
[im56@cobalt Irsim]$ calc_poly_64.pl soccer_stars_top_flat.mag
calculating total polysilicon area...
================================================
Total PolySilicon Area = 7030320 lambda squared
given 6800 x 6800 area, total poly density = 15.20398 percent
Local polysilicon density = 15.20398 percent
================================================
[im56@cobalt Irsim]$ calc_metal1_64.pl soccer_stars_top_flat.mag
calculating total metal1 area...
================================================
Total Metal1 Area = 13916523 lambda squared
given 6800 x 6800 area, total metal1 density = 30.09629 percent
================================================
[im56@cobalt Irsim]$ calc_metal2_64.pl soccer_stars_top_flat.mag
calculating total metal2 area...
================================================
Total Metal2 Area = 14289762 lambda squared
given 6800 x 6800 area, total metal2 density = 30.90346 percent
================================================
```

Figure 8: Densities are all above their respective thresholds.

Verification and Testing

We performed testing for each important module of the chip separately, going through a pre and post Design Compiler flow for the FSM controller, datapath and padframe integration. In addition, for the fully integrated circuit, we performed detailed verification in IRSIM using the .sim file generated by the flattened Magic layout. The test case used for the IRSIM simulation was Test Case 1. For the individual module testing and verification, we used input patterns that should test both standard operation and edge cases.

There are several test cases that we chose to verify the response of the circuit to varying input frequencies and time frames, ranging from inputs in perfect time succession, spread out inputs in slow succession and inputs with random timing.

The two modules (FSM controller and datapath) and the integrated design performed as expected with our testbenches, showing no timing or logical errors in their outputs under the comprehensive test cases mentioned above. There are no differences between the pre and post Design Compiler simulation waveforms. Below we explain each test case we utilized, and then we present the annotated simulation results for all modules and the final design. As mentioned, only the final integrated layout has an IRSIM simulation, which features Test Case 1, and the testing is being done directly from the padframe in that instance.

Test Cases for Full Integration

- Test 1: Sequential, evenly-spaced button presses (thinking times: 1, 2, 3, 4 cycles). This is the baseline "standard operation" case with short, ordered delays, verifying that the FSM correctly transitions through all four LOAD states, latches distinct ring oscillator values for each parameter, completes the math/collision pipeline, and asserts valid with correct ball_x/y, keeper_x/y, and win_loss outputs.
- Test 2: Irregular, out-of-order thinking times (5, 1, 6, 2 cycles). The asymmetric delays ensure the ring counters are sampled at substantially different phases than in Shot 1, verifying that the latched strike_power, spin, v_angle, and h_angle values differ from the previous shot and produce a distinct trajectory and outcome, confirming that the randomization mechanism is functioning correctly.
- Test 3: Zero-delay "fast press" case (0, 0, 0, 0 cycles). All four ctrl pulses are asserted as soon as each LOAD state is entered, so the ring counters are sampled at their initial post-restart values. This is an edge case that checks the circuit does not require any minimum inter-press gap and that back-to-back button presses do not cause setup violations, missed transitions, or metastability in the fully integrated circuit.
- Test 4: Long, irregular thinking times (7, 3, 9, 5 cycles). The extended delays exercise the FSM's ability to hold each LOAD state indefinitely while waiting for a ctrl pulse, confirming there is no timeout or state corruption during prolonged idle periods between presses, and that the final computed result is still valid after the longest inter-press intervals.

FSM Controller

Simulation Results (Questa pre Design Compiler)

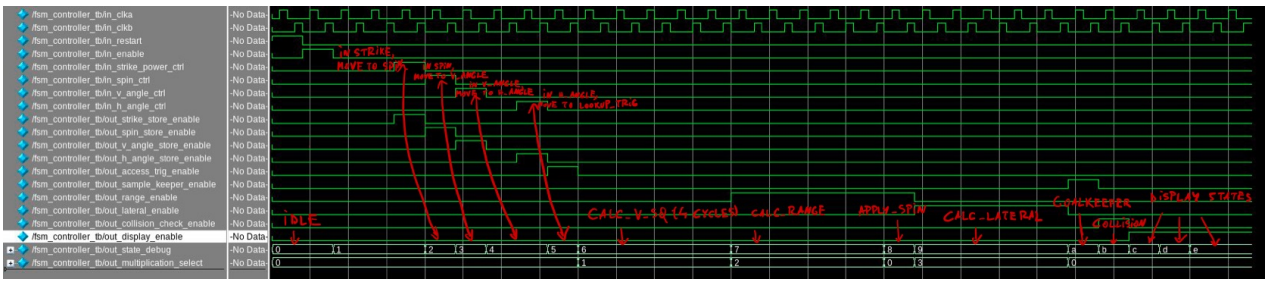

Figure 9: FSM controller simulation waveform pre Design Compiler

Simulation Results (Questa post Design Compiler)

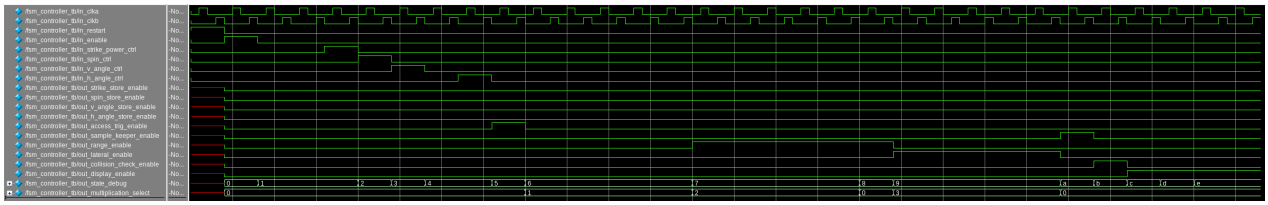

Figure 10: FSM controller simulation waveform post Design Compiler

Datapath

Simulation Results (Questa pre Design Compiler)

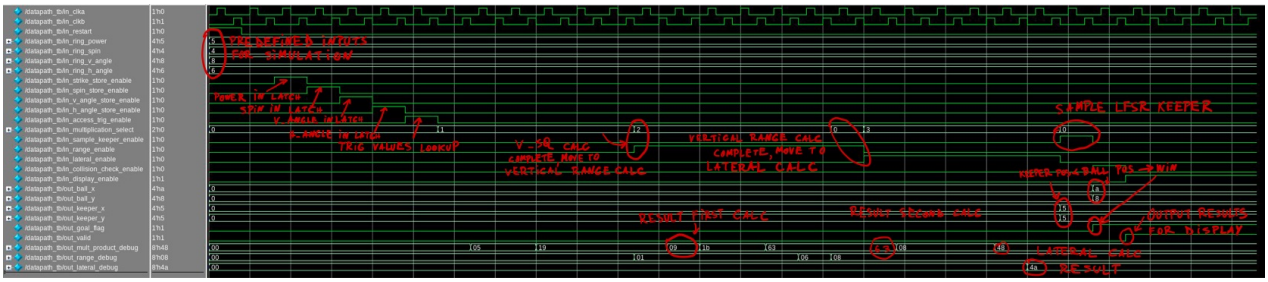

Figure 11: Datapath simulation waveform pre Design Compiler

Simulation Results (Questa post Design Compiler)

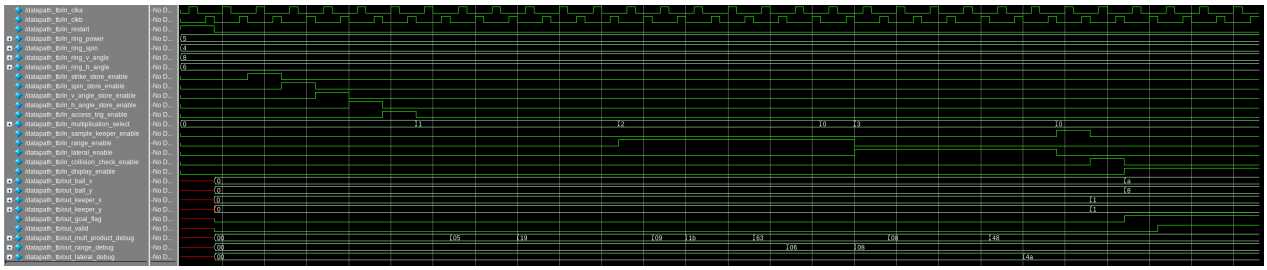

Figure 12: Datapath simulation waveform pre Design Compiler

Padframe Integration

Simulation Results (Questa pre Design Compiler)

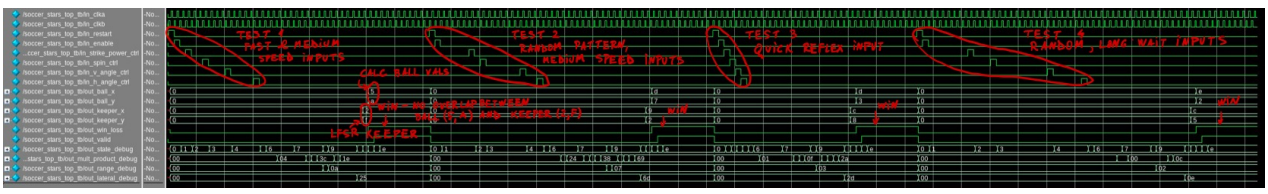

Figure 13: Padframe integrated simulation waveform pre Design Compiler

Simulation Results (Questa post Design Compiler)

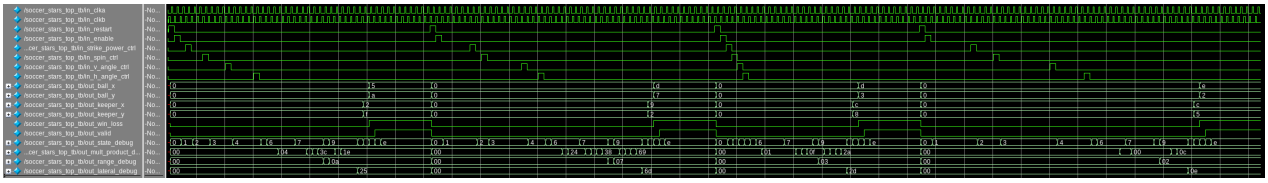

Figure 14: Padframe integrated simulation waveform post Design Compiler

Simulation Results (IRSIM post place and route)

Figure 15: Padframe integrated Irsim simulation, Test Case 1

Results and Lessons Learned

The chip met almost all specifications that we had originally proposed. We were able to successfully handle complex logic operations such as the LFSR based pseudo-number generation, the ring oscillators and bit shift multiplication. The core had 54 pins and fit nicely into a 64-pin padframe and yielded favorable final results. Below are several of these results as given by Design Compiler.

Core Performance Summary

Metric	Value	Notes
Slack	4.62	Slack is MET after Design Compiler synthesis
Approximate area	2300 $\lambda \times$ 2300 λ	Approximate area of the core
Power	10.9265 mW	Total power consumption
Cell count	859	Number of synthesized cells by Design Compiler
Transistor count	7158	Total number of p-type (3579) and n-type (3579) transistors
Total cell area	310338 λ^2	Area occupied by all cells

Lessons Learned

- Cut down on unimportant pins to save time while routing and reduce padframe size if possible.
- Discuss FSM and datapath design on a broader level before actually writing any code.
- Errors will come up regularly, thus debugging, and doing so effectively, is a crucial skill.

Conclusion and Future Testing

Future Improvements

A possible future task would be to implement score-keeping into the game. This would add a significant number of pins but also add an extra layer of complexity, as instead of single-round gameplay the game would contain a continuous sequence of rounds that all contribute to a running total.

It would also be wise to increase the size of the goalkeeper to make saves more likely. Currently the collision check uses a tolerance of ± 2 on each axis, giving the keeper a 5×5 save zone on the 16×16 grid. Increasing this tolerance to ± 3 or ± 4 would expand the save area and balance the game, since under the current rules the player wins the vast majority of rounds.

Testing Strategy

A possible future testing strategy would use a 16×16 LED matrix and a microcontroller such as an ESP32 to handle display operations. The chip would remain the brain of the game, computing all necessary values and handling input latching, while the ESP32 would serve as the display driver and button interface. Five push buttons would handle input latching with no alterations to the chip itself. We are presenting the proposed wiring diagram and parts list are presented below, along with an explanation of the connections.

Testing Parts List

Part	Amount	Function
ASIC	1	Computational brain of the game
ESP32	1	Display driver and button interface
16×16 Neopixel LED matrix	1	Displays ball and keeper positions
Push buttons	5	Latch input values (power, spin, v_angle, h_angle, restart)
AMS1117-3.3 regulator	1	Steps 5V down to 3.3V for the ASIC and ESP32
USB-C breakout	1	Provides 5V supply
1000 μF capacitor	1	Smooths the 5V matrix rail
470 Ω resistor	1	Series resistor on the matrix data line

Connections Overview

The system is powered through a USB-C connector, which supplies 5 V directly to the LED matrix and to the input of an AMS1117-3.3 regulator. The regulator generates the 3.3 V rail used by both the ESP32 and the ASIC. All grounds are tied to a common node. We assumed that the ASIC will be powered by 3.3 V; if this not the case, another AMS1117 module can be used to step down to 1.8 V or whatever voltage necessary.

The ESP32 acts as the bridge between the player and the chip. It reads the five debounced push buttons on its GPIO pins and drives the ASIC's control inputs (strike_power_ctrl, spin_ctrl, v_angle_ctrl, h_angle_ctrl, and restart) accordingly. The enable pin is driven high in software so the chip is active whenever powered. The ESP32 also generates the two non-overlapping clocks clka and clkb using its PWM peripheral, eliminating the need for external oscillators.

On the output side, the ASIC's ball_x, ball_y, keeper_x, keeper_y, win_loss, and valid pins are wired to ESP32 GPIOs. When valid is asserted, the ESP32 latches the coordinates and renders the frame to the WS2812B matrix over a single data line through a 470 Ω series resistor. The keeper is drawn as a 5×5 red patch matching the collision tolerance, and the ball is drawn as a 2×2 patch centered on its coordinates, green on a goal, yellow on a save, so the player can see both the save zone and the shot result. This is very close to our game's original intended display method.

Testing Wiring Diagram

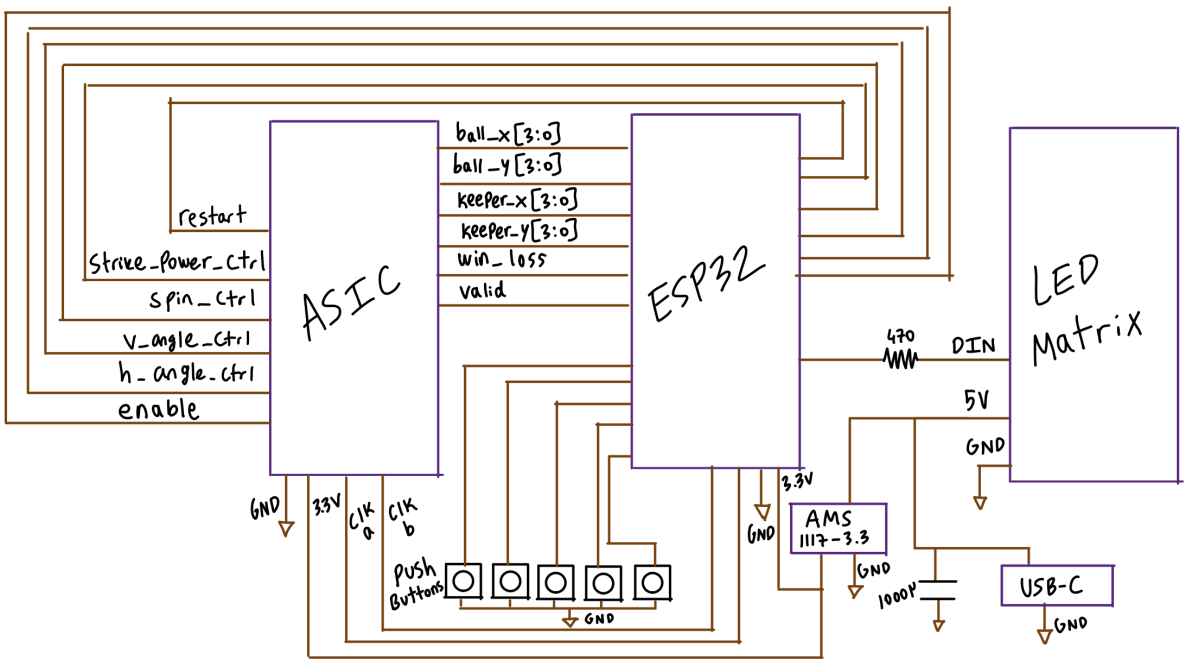

Figure 16: Wiring diagram with ESP32 and LED matrix.

Appendix

A. Files

All the required code and compilation files will be submitted to the final project checklist assignment.

B. References

Dr. Joseph Cavallaro, Professor in the Department of Electrical and Computer Engineering, Rice University